

INVESTIGATION REPORT

Issue 1010: Debug single-step inside a hardware loop

openhwgroup/cv32e40p

|                |                                                         |
|----------------|---------------------------------------------------------|
| Document       | RTL-TRIAGE-CV32E40P-1010                                |
| Disposition    | Dead code, proven                                       |
| Record date    | 2026-09-09                                              |
| Classification | Internal working document. Not a verification sign-off. |

1 Status

|               |                                                   |
|---------------|---------------------------------------------------|
| Issue         | openhwgroup/cv32e40p#1010                         |
| Issue filed   | 2024-06-24                                        |
| Issue type    | Coverage hole referred for disposition            |
| Affects       | cv32e40p                                          |
| Configuration | COREV_PULP=1 FPU=0                                |
| Disposition   | Dead code, proven                                 |
| Severity      | Not assessed                                      |
| Agreement     | Same disposition as the independent human record. |
| Evidence      | 11 cited, 10 present, 1 not retained              |

Disposition meaning. The code cannot be reached in any state permitted by the stated model and assumptions, established by an unbounded proof.

Severity is not recorded for this run. The harness does not infer a severity from the disposition, so the field is reported as not assessed rather than filled in.

2 Description

The branch at cv32e40p\_controller.sv:850 (debug\_single\_step\_i & ~debug\_mode\_q inside DECODE\_HWLOOP) and its dependent lines 852-887 are unreachable. An unbounded proof (abc pdr, top=cv32e40p\_core, COREV\_PULP=1) shows that no reachable state has ctrl\_fsm\_cs==DECODE\_HWLOOP together with debug\_single\_step\_i=1 and debug\_mode\_q=0; a second proof of the exact line-850 guard (including instr\_valid\_i, no pending debug request/trigger, no IRQ) also passes. The enclosing DECODE\_HWLOOP branch itself is reachable (bounded cover hits it at step 8), so the proofs are not vacuous. Mechanism (corrected 2026-09-24): dcsr.step changes only through dret, which leaves debug mode via XRET\_JUMP into DECODE, or through a CSR write, which is a csr\_status instruction and flushes the pipeline back to DECODE. In DECODE with step=1 the line-664 logic halts IF and redirects to DBG\_FLUSH before DECODE\_HWLOOP can be entered, and the proof confirms the id\_ready\_i=0 corner does not open a path. An earlier version of this report credited the decoder rule that dcsr is writable only in debug mode; negative controls show that rule is not what closes the hole: making dcsr writable outside debug mode still passes, while also removing the flush on that write gives a counterexample. The verdict is unchanged. Condition coverage (added 2026-09-24): line 850 has four condition rows. Rows 2 and 3 (debug\_single\_step\_i=1 with debug\_mode\_q=0) are the unreachable case proven here. Row 4 (debug\_single\_step\_i=1 with debug\_mode\_q=1) is reachable and is now hit: a formal cover reaches it in 12 cycles, and a directed Verilator test (chips/cv32e40p/tests/hole\_1010\_row4/portable: in debug mode, set dcsr.step and run a hardware loop; runs on the stock core-v-verif testbench) hits it on 28 cycles, against 0 in a control run that is identical except that dcsr.step is never set. Row 1 is hit by the existing regression. With this, every part of the hole is either proven unreachable or covered. Environment assumptions: OBI handshake legality, scan\_cg\_en\_i=0, fetch\_enable\_i=1.

### 3 Conditions

1. OBI handshake legality
2. scan\_cg\_en\_i=0
3. fetch\_enable\_i=1

*Extracted verbatim from the environment assumptions stated in the verdict record for this run.*

### 4 Implications

*Not recorded as a separate field for this run. What the disposition means for a user of the core follows from the Disposition meaning in Status and the bounds in Limitations and scope.*

### 5 Action

#### Waiver text, to be applied to the coverage exclusion

cv32e40p\_controller.sv:850-887 (DECODE\_HWLOOP single-step branch): formally proven unreachable. dcsr.step changes only through dret (XRET\_JUMP -> DECODE) or a CSR write (pipeline flush back to DECODE); in DECODE with step=1 the controller halts IF and routes to DBG\_FLUSH/FLUSH\_EX/DBG\_WAIT\_BRANCH, so DECODE\_HWLOOP is never entered with step=1 outside debug mode. Proof: runs/20260909-1238-cv32e40p-1010/formal/p\_assert (abc pdr, unbounded) + anti-vacuity cover c\_state. Exclude the line-850 true branch, condition rows 2 and 3 (step=1 with debug\_mode\_q=0), and lines 852-887 from coverage. Do NOT exclude condition row 4 (step=1 with debug\_mode\_q=1): it is reachable and is covered by the directed test chips/cv32e40p/tests/hole\_1010\_row4/portable.

#### Checker used

checker\_1010.sv

### 6 Evidence

| Ref | Type           | Path, relative to the run directory                   | File     | Result recorded                                                                                                                                                                                                                             |
|-----|----------------|-------------------------------------------------------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| E1  | proof          | formal/p_assert/sby.log                               | present  | abc pdr PASS: exact guard of line 850 never true in any reachable state                                                                                                                                                                     |
| E2  | proof          | formal/p_strong/sby.log                               | present  | abc pdr PASS: DECODE_HWLOOP never coexists with debug_single_step_i=1 & debug_mode_q=0                                                                                                                                                      |
| E3  | proof          | formal/c_state/sby.log                                | present  | smtbmc cover PASS: DECODE_HWLOOP state and the exact enclosing branch reached at step 8 (covers_reached=2) -> asserts not vacuous                                                                                                           |
| E4  | counterexample | formal/c_state/job/engine_0/trace1.vcd                | not kept | cover witness trace: entry into DECODE_HWLOOP via hwloop body; rendered in cover_decode_hwloop.png                                                                                                                                          |
| E5  | sim-log        | sim/hwloop_dt/sim.log                                 | present  | Directed test hwloop_dt (real cv.setupi hardware loop, 10 iterations) on the core-v-verif Verilator testbench: EXIT SUCCESS, acc=30, controller observed in DECODE_HWLOOP in simulation with debug_single_step_i=0 (sim_hwloop_witness.png) |
| E6  | proof          | ../upstream-pr-1010/jobs/N_negctl_pulp_fpu0_prove.out | present  | negative control, dcsr writable outside debug mode: abc pdr still PASS, so that rule is not the protection                                                                                                                                  |
| E7  | proof          | ../upstream-pr-1010/jobs/N2_negctl_noflush_prove.out  | present  | negative control, dcsr writable outside debug mode and no flush: abc pdr FAIL with counterexample                                                                                                                                           |
| E8  | proof          | ../upstream-pr-1010/jobs/P_pulp_fpu0_prove.out        | present  | recheck on upstream dev with upstream's own assumptions (no fetch_enable assumption): abc pdr PASS                                                                                                                                          |
| E9  | cover          | ../upstream-pr-1010/jobs/R4b_row4_valid_cover.out     | present  | condition row 4 reachable: DECODE_HWLOOP, instr_valid_i, step=1, debug_mode_q=1 reached at step 12                                                                                                                                          |
| E10 | simulation     | ../cv32e40p-1010-row4/p_step/sim.log                  | present  | portable directed test, stock testbench (no hex rewrite): EXIT SUCCESS, row 4 hit on 28 cycles, rows 2/3 on 0                                                                                                                               |
| E11 | simulation     | ../cv32e40p-1010-row4/p_nostep/sim.log                | present  | portable control (dcsr.step never set): EXIT SUCCESS, row 4 hit on 0 cycles                                                                                                                                                                 |

Paths are relative to runs/20260909-1238-cv32e40p-1010. The File column is a check made when this document was generated. It reports whether the cited file exists, not whether its contents were re-validated. 'not kept' means a repository rule deliberately excludes that artefact, usually a waveform dump; rerun the check to regenerate it. 'MISSING' means the file is absent and was expected to be there.

What each class of artefact establishes

- proof. Proof or engine run. Scope depends on the mode: prove is unbounded under the stated model, bmc is bounded to its depth, cover establishes reachability only.
- counterexample. A concrete input sequence produced by the engine. Shows the stated condition is reachable; it is not itself a proof.
- sim-log. Simulation. Exercises the stimulus that was run. It does not establish an unbounded proof.

Artefact(s) E4 are not retained under repository policy. The result recorded against them stands on the engine log cited alongside; the artefact itself must be regenerated by rerunning the check before it can be inspected.

7

Limitations and scope

- The proof establishes unreachability under the model and the environment assumptions named in Status and Conditions. It does not establish that the surrounding logic is correct.
- This is an internal investigation record. It requires engineering review before it is used as a disposition. No report generator can supply missing evidence.
- Results apply to the configuration and core revision named in Status. A different configuration needs a separate run.
- Public demonstration runs on open cores are not client engagements and not production verification results.

8

Provenance and measurement

|                     |                                                                                                                                |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Run directory       | runs/20260909-1238-cv32e40p-1010                                                                                               |
| Title source        | Title and configuration read back from reports/issues/cv32e40p_issue_1010.pdf, the previously published report for this issue. |
| Inputs shown        | title, body(links stripped), labels                                                                                            |
| Human record        | Avinash 2026-09: yosys/abc PDR unbounded proof, controller never in DECODE_HWLOOP while single-stepping outside debug mode     |
| Assessed confidence | 0.9. This is the investigator's own estimate. It is not a measurement and not a reliability figure.                            |
| Iterations          | 1                                                                                                                              |

Generated by reporting/scripts/issue\_document.py from runs/20260909-1238-cv32e40p-1010/verdict.json. The generator typesets recorded data and checks that cited files exist. It does not run, rerun or validate any check, and it cannot supply missing evidence.